

model/d2q.py (Q&A Documentation)

Q: What does this file implement at a high level? **A:** This file implements the D2Q baseline: it predicts a normalized quantile position for watch-time within a duration bucket, rather than predicting watch-time directly.

Q: Why is the duration bucket feature important for D2Q? **A:** Watch-time is constrained by video duration. Conditioning on a duration bucket maps labels to a more comparable scale within each bucket and yields a smoother target.

Q: What does this file export (public API)? **A:** The public API is the set of classes/functions other modules import (sometimes re-exported by package initializers). The key top-level definitions detected here are:

```
class D2Q # methods: __init__, build, forward
```

Q: What are the main dependencies (imports) and why do they matter? **A:** Imports show whether this file is model code (torch), data code (pandas), or evaluation/analysis (sklearn). They also determine runtime requirements.

```
import torch
```

```
from model.layers import MultiLayerPerceptronD2Q
```

```
class D2Q(torch.nn.Module):
```

```
    def __init__(self, description, embed_dim, mlp_dims, dropout):
```

```
        super().__init__()
```

```
        self.features = {name: (size, type) for name, size, type in description if (type in ["ctn", 'seq'])}
```

```
        self.build(embed_dim, mlp_dims, dropout)
```

```
    def build(self, embed_dim, mlp_dims, dropout):
```

```
        self.emb_layer = torch.nn.ModuleDict()
```

```
        self.ctn_emb_layer = torch.nn.ParameterDict()
```

```
        self.ctn_linear_layer = torch.nn.ModuleDict()
```

```
        embed_output_dim = 0
```

```
        for name, (size, type) in self.features.items():
```

```
            if type == 'spr':
```

```
                self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
```

```
                embed_output_dim += embed_dim
```

```
            elif type == 'ctn':
```

```
                self.ctn_linear_layer[name] = torch.nn.Linear(1, 1, bias=False)
```

```
                embed_output_dim += 1
```

```
            elif type == 'seq':
```

```

        self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
        embed_output_dim += embed_dim
    else:
        raise ValueError('unkown feature type: {}'.format(type))
    self.mlp = MultiLayerPerceptronD2Q(embed_output_dim, mlp_dims, dropout)
    return

# def init(self):
#     for param in self.parameters():
#         torch.nn.init.uniform_(param, -0.01, 0.01)

def forward(self, x_dict):
    linears = []
    embs = []
    for name, (_, type) in self.features.items():
        x = x_dict[name]
        if type == 'spr':
            embs.append(self.emb_layer[name](x).squeeze(1))
        elif type == 'ctn':
            linears.append(self.ctn_linear_layer[name](x))
        elif type == 'seq':
            seq_emb = self.emb_layer[name](x)
            seq_mask = torch.unsqueeze(x_dict["{}mask".format(name)], dim=2)
            embs.append(torch.sum(seq_emb * seq_mask, dim=1) / torch.sum(seq_mask, dim=1))
        else:
            raise ValueError('unkwon feature: {}'.format(name))

```

Q: What is D2Q and what responsibility does it have? **A:** This class is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What are the important methods on D2Q? **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```

__init__
build
forward

```

Q: What does the implementation look like for D2Q? **A:** Excerpt from the file:

```

class D2Q(torch.nn.Module):

    def __init__(self, description, embed_dim, mlp_dims, dropout):
        super().__init__()
        self.features = {name: (size, type) for name, size, type in description if (type in ["ctn", 'seq', 'spr'])}
        self.build(embed_dim, mlp_dims, dropout)

    def build(self, embed_dim, mlp_dims, dropout):
        self.emb_layer = torch.nn.ModuleDict()
        self.ctn_emb_layer = torch.nn.ParameterDict()
        self.ctn_linear_layer = torch.nn.ModuleDict()
        embed_output_dim = 0
        for name, (size, type) in self.features.items():
            if type == 'spr':
                self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
                embed_output_dim += embed_dim
            elif type == 'ctn':

```

```

        self.ctn_linear_layer[name] = torch.nn.Linear(1, 1, bias=False)
        embed_output_dim += 1
    elif type == 'seq':
        self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
        embed_output_dim += embed_dim
    else:
        raise ValueError('unkown feature type: {}'.format(type))
self.mlp = MultiLayerPerceptronD2Q(embed_output_dim, mlp_dims, dropout)
return

# def init(self):
#     for param in self.parameters():
#         torch.nn.init.uniform_(param, -0.01, 0.01)

def forward(self, x_dict):
    linears = []
    embs = []
    for name, (_, type) in self.features.items():
        x = x_dict[name]
        if type == 'spr':
            embs.append(self.emb_layer[name](x).squeeze(1))
        elif type == 'ctn':
            linears.append(self.ctn_linear_layer[name](x))
        elif type == 'seq':
            seq_emb = self.emb_layer[name](x)
            seq_mask = torch.unsqueeze(x_dict["{}mask".format(name)], dim=2)
            embs.append(torch.sum(seq_emb * seq_mask, dim=1) / torch.sum(seq_mask, dim=1))
        else:
            raise ValueError('unkwon feature: {}'.format(name))
    emb = torch.concat(embs + linears, dim=1)
    res = self.mlp(emb)
    res = res.squeeze(1)
    return torch.sigmoid(res)

```

Q: Why is D2Q needed by training scripts? **A:** Training scripts assume this class can be instantiated from the dataset description and can map a feature dictionary to outputs required by that script's loss. This separation makes it easy to swap models while reusing the same dataloader and metrics.

Q: Example: end-to-end data flow involving this file **A:** Core pipeline shape (schematic):

```

for features, label in dataloaders["train"]:
    out = model(features)
    loss = ...
    loss.backward(); optimizer.step()

```

Q: Full source code of the Python file **A:** The complete file is included verbatim for reference.

```

import torch

from model.layers import MultiLayerPerceptronD2Q

class D2Q(torch.nn.Module):

    def __init__(self, description, embed_dim, mlp_dims, dropout):
        super().__init__()
        self.features = {name: (size, type) for name, size, type in description if (type in ["ctn", 'seq', 'spr'])}
        self.build(embed_dim, mlp_dims, dropout)

    def build(self, embed_dim, mlp_dims, dropout):
        self.emb_layer = torch.nn.ModuleDict()
        self.ctn_emb_layer = torch.nn.ParameterDict()

```

```

self.ctn_linear_layer = torch.nn.ModuleDict()
embed_output_dim = 0
for name, (size, type) in self.features.items():
    if type == 'spr':
        self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
        embed_output_dim += embed_dim
    elif type == 'ctn':
        self.ctn_linear_layer[name] = torch.nn.Linear(1, 1, bias=False)
        embed_output_dim += 1
    elif type == 'seq':
        self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
        embed_output_dim += embed_dim
    else:
        raise ValueError('unkown feature type: {}'.format(type))
self.mlp = MultiLayerPerceptronD2Q(embed_output_dim, mlp_dims, dropout)
return

# def init(self):
#     for param in self.parameters():
#         torch.nn.init.uniform_(param, -0.01, 0.01)

def forward(self, x_dict):
    linears = []
    embs = []
    for name, (_, type) in self.features.items():
        x = x_dict[name]
        if type == 'spr':
            embs.append(self.emb_layer[name](x).squeeze(1))
        elif type == 'ctn':
            linears.append(self.ctn_linear_layer[name](x))
        elif type == 'seq':
            seq_emb = self.emb_layer[name](x)
            seq_mask = torch.unsqueeze(x_dict["{}mask".format(name)], dim=2)
            embs.append(torch.sum(seq_emb * seq_mask, dim=1) / torch.sum(seq_mask, dim=1))
        else:
            raise ValueError('unkwon feature: {}'.format(name))
    emb = torch.concat(embs + linears, dim=1)
    res = self.mlp(emb)
    res = res.squeeze(1)
    return torch.sigmoid(res)

```